

Disentangled Graph Prompting for Out-Of-Distribution Detection

Cheng Yang^{ID}, Yu Hao^{ID}, Qi Zhang^{ID}, and Chuan Shi^{ID}, *Senior Member, IEEE*

Abstract—When testing data and training data come from different distributions, deep neural networks (DNNs) will face significant safety risks in practical applications. Therefore, out-of-distribution (OOD) detection techniques, which can identify OOD samples at test time and alert the system, are urgently needed. Existing graph OOD detection methods usually characterize fine-grained in-distribution (ID) patterns from multiple perspectives, and train end-to-end graph neural networks (GNNs) for prediction. However, due to the unavailability of OOD data during training, the absence of explicit supervision signals could lead to sub-optimal performance of end-to-end encoders. To address this issue, we follow the *pre-training+prompting* paradigm to utilize pre-trained GNN encoders, and propose Disentangled Graph Prompting (DGP), to capture fine-grained ID patterns with the help of ID graph labels. Specifically, we design two prompt generators that respectively generate class-specific and class-agnostic prompt graphs by modifying the edge weights of an input graph. We also design several effective losses to train the prompt generators and prevent trivial solutions. We conduct extensive experiments on ten datasets to demonstrate the superiority of our proposed DGP, which achieves a relative AUC improvement of 3.63% over the best graph OOD detection baseline. Ablation studies and hyper-parameter experiments further show the effectiveness of DGP.

Index Terms—Graph neural networks, out-of-distribution detection.

I. INTRODUCTION

TRADITIONAL deep neural networks (DNNs) typically operate under the assumption that the data used during model training and testing phases are independent and identically distributed. Despite the significant potential of DNNs in various domains, they still face challenges in practical applications [1], e.g., misclassifying out-of-distribution (OOD) samples that deviate significantly from the training data distribution [2],

[3]. Therefore, timely detecting OOD samples and alerting the system to take appropriate preventive measures in unfamiliar situations is crucial to ensuring the system's security. In recent years, there has been extensive research on OOD detection tasks in the fields of vision [4], [5] and language [6], [7].

Recently, OOD detection has extended its reach to graph data. To address the graph OOD detection problem, existing methods typically model in-distribution (ID) patterns from multiple granularities or aspects, and train an end-to-end GNN for prediction. For example, GOOD-D [8] introduces a graph contrastive learning method with node-, group- and graph-level discrepancy modeling. GraphDE [9] utilizes the class label information of ID graphs, and models class-specific graph generation processes to characterize ID patterns. However, since OOD data is unavailable at the training phase, the lack of direct supervision signals can result in the sub-optimal capability of end-to-end encoders [10]. To this end, AAGOD [10] proposes a post-hoc manner to leverage pre-trained graph encoders for OOD detection. The pre-trained encoders can benefit from self-supervised learning, and thus perform better than the aforementioned end-to-end ones. In addition to these methods, SEGO [11] enhances OOD detection by minimizing structural entropy to capture essential graph patterns, while HGOE [12] further improves robustness through hybrid external and internal outlier exposure. Finally, GOODAT [13] offers a data-centric, test-time solution using a graph masker optimized by GIB-boosted losses, providing a versatile approach for detecting OOD samples without modifying the original GNN architecture. Inspired by prompt learning, we aim to keep pre-trained encoders frozen, while tailor input graphs as prompt graphs so that the pre-trained encoders can adapt to the OOD detection task. Furthermore, we aim to integrate the advantages of fine-grained ID pattern modeling in previous work for better OOD detection performance. Nevertheless, how to generate prompt graphs from different views for fine-grained ID pattern mining, is still non-trivial.

In this paper, we make the first attempt to combine the strengths of fine-grained ID pattern modeling and the *pre-training+prompting* framework for OOD detection. We introduce a novel Disentangled Graph Prompting (DGP) method for OOD detection, as shown in Fig. 1. Our goal is to discern both class-specific and class-agnostic ID patterns using ID graph labels. The class-specific pattern mainly comprises discriminative information for distinguishing ID classes, whereas the class-agnostic pattern encompasses shared information among ID samples. Both of these patterns contain crucial cues for OOD detection. Specifically, we pursue the following strategies: (1) We pre-train a GNN encoder by typical self-supervised learning methods [14], [15], and freeze its parameters. (2) We design two prompt graph generators that manipulate the edge weights of the original graph to generate class-specific and class-agnostic

Received 14 December 2023; revised 4 November 2025; accepted 19 March 2026. Date of publication 2 April 2026; date of current version 2 June 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62550138, Grant 62192784, Grant 62572064, and Grant 62472329, in part by Young Elite Scientists Sponsorship Program under Grant 2023QNRC001 by CAST, and in part by Beijing Natural Science Foundation under Grant 253004. Recommended for acceptance by L. Chen. (*Corresponding author: Chuan Shi.*)

Cheng Yang, Yu Hao, and Chuan Shi are with the Beijing Key Lab of Intelligent Telecommunications Software and Multimedia, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: yangcheng@bupt.edu.cn; haoyuu@bupt.edu.cn; shichuan@bupt.edu.cn).

Qi Zhang is with China Mobile Group Shaanxi Company, Ltd., Xi'an 710072, China (e-mail: zhangqi3@sn.chinamobile.com).

Code is available at <https://github.com/BUPT-GAMMA/DGP>.

This article has supplementary downloadable material available at <https://doi.org/10.1109/TKDE.2026.3678022>, provided by the authors.

Digital Object Identifier 10.1109/TKDE.2026.3678022

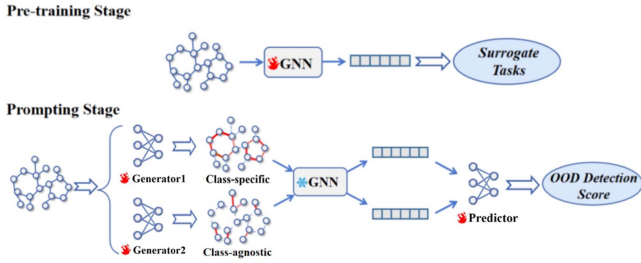


Fig. 1. An illustration of our Disentangled Graph Prompting (DGP) method. DGP can take advantage of pre-trained GNN encoders, and generate prompt graphs from both class-specific and class-agnostic views for fine-grained ID pattern mining. The parameters of GNN encoders are frozen at the prompting stage.

prompts. (3) Feeding the prompt graphs into pre-trained GNN encoder and a predictor, DGP transforms prompts into class distributions, and introduces class-specific and class-agnostic losses to supervise the training of two prompt generators. The losses encourage the prompt graphs to emphasize the key patterns within ID graphs, thereby enhancing the distinction between ID and OOD graphs. (4) We also incorporate a regularization term to prevent the learnable prompt graphs from falling into trivial solutions.

We conduct extensive experiments on real-world datasets from molecular, social network and bioinformatics domains to validate the performance of DGP. We utilize two self-supervised methods, i. e., GCL [14] and SimGRACE [15], to pre-train the GNN encoder. Experimental results reveal that:

- 1) Non-graph OOD methods perform poorly on graph data (average AUC 48.95%), confirming the need for structural modeling.
- 2) DGP improves AUC by 13.65% over fine-tuned GNNs and 3.63% over the best SOTA baseline, demonstrating its effectiveness.
- 3) Additional experiments, such as ablation studies, encoder initialization analysis and efficiency evaluation, further confirm DGP's robustness, interpretability, and scalability.

Our contributions can be summarized as follows:

- To the best of our knowledge, we are the first to comprehensively characterize fine-grained ID patterns within the novel *pre-training+prompting* paradigm tailored for graph OOD detection. This innovative idea sheds light on previously unexplored aspects of graph OOD detection.
- We propose a novel method named DGP for graph OOD detection, which generates class-specific and class-agnostic prompt graphs to capture ID patterns, and introduces several effective losses to supervise prompt generators and prevent trivial solutions.
- Experimental results on real-world datasets show that DGP achieves 3.63% relative AUC improvement on average over SOTA OOD detection methods. Extensive experiments further demonstrate the effectiveness of our design.

II. RELATED WORK

A. Graph Neural Networks

Graph data is everywhere in real-life scenarios, where entities and their interrelationships in various systems can be represented as graph-structured data for in-depth analysis [16], [17], [17], [18]. GNNs leverage structural information in graph data by the

message-passing mechanism, and have been widely applied in tasks such as node classification, link prediction, recommendation systems, and molecular property prediction [19], [20], [21].

GNN encoders can be categorized into two types: spectral-based and spatial-based ones. Spectral-based GNNs inherit principles from graph signal processing, defining graph convolution operations in the spectral domain. For example, SCNN [22] employed a learnable diagonal matrix to replace the spectral-domain convolutional kernel, facilitating graph convolution operations. ChebNet [23] utilized Chebyshev polynomials to approximate K-order localized graph filters, further enhancing efficiency. In contrast, spatial-based GNNs directly operate over adjacent nodes and perform message passing. For example, GCN [24] proposed the aggregation of node features from one-hop neighbors and aggregating neighbor information along the topological structure. GraphSAGE [25] randomly sampled a fixed number of neighbors, and designed diverse aggregation methods. GAT [26] introduced an attention mechanism to assign distinct weights to different neighbors. APPNP [27] performed feature transformation operations before neighbor aggregation.

Recently, self-supervised methods [28], [29], [30] have surpassed supervised ones in various tasks, demonstrating significant potential in real-world applications. Graph contrastive learning, a representative self-supervised technique, allows GNNs to get pretrained without labels through context information or auxiliary tasks. These methods typically generate data augmentations first, and then maximize/minimize the similarities between positive/negative instances. GraphCL [14] incorporated four types of data augmentation strategies to generate augmentation views. SimGRACE [15] introduced Gaussian noise perturbation of model parameters in the graph encoder to construct an augmentation view. DGI [31] aimed to maximize local mutual information, intending to learn local node features capable of capturing global graph information. GRACE [32] generated graph views by removing edges and masking node features, and then maximizes the agreement of node embeddings in these two augmentation views.

B. Graph Prompt Learning

Although graph pre-training [33], [34], [35], [36] has emerged as a powerful paradigm in recent years, there is usually an inherent gap between the tasks used for pre-training and the objectives of downstream tasks, which limits the extensive use of pre-trained models [37]. In the field of Natural Language Processing (NLP), appropriate prompts can effectively narrow the gap between pre-trained models and downstream tasks, enabling them to handle various downstream tasks [38], [39], [40], [41]. Recently, the idea of prompting was also introduced to graph learning.

Existing work can be divided into two categories. The first category aims to fast adapt to multiple tasks via graph prompting. [42] designed prompt tokens, token structures, and embedding patterns, unifying the format of NLP prompts and graph prompts. Meta-learning was also employed to ensure a dependable initial prompt. GRAPH-PROMPT [43] set the link prediction task as the pre-training task, and utilized node classification and graph classification tasks as downstream tasks. A learnable prompt was designed as the parameters in the ReadOut operation. OFA [44] introduced a graph prompting paradigm (GPP) that incorporated a prompt graph into the original input graph in a way customized for the specific task. The nodes

TABLE I
COMPARISON AMONG GRAPH OOD DETECTION WORK

Method	Fine-grained modeling	No OOD exposure	Pre-trained encoder utilized
OCGIN			
GLocalKD	✓		
GraphDE	✓	✓	
GOOD-D	✓	✓	
AAGOD		✓	✓
DGP	✓	✓	✓

within the prompt graph encompassed all relevant information pertaining to the downstream task. The second category focuses on adapting a specific task with few-shot or even zero-shot samples. GPPT [45] designed the graph promoting function to modify the standalone node into token pairs, and transformed the downstream node classification tasks into link prediction tasks for reconstruction purposes. SGL-PT [46] designed a promoting function that introduced a masked super node into a single graph. Then, the original classification task was transformed into reconstructing the super node's representations. AAGOD [10] learned graph amplifiers as prompts to capture ID graph patterns, converting a pre-trained GNN encoder from graph classification to OOD detection.

C. Graph Out-of-Distribution Detection

Graph OOD detection methods can be divided into node-level and group-level ones. For node-level methods, GPN [47] introduced a Bayesian update with the help of density estimation and diffusion to identify OOD nodes. GKDE [48] developed a multi-source uncertainty framework for detecting OOD nodes. GNNsafe [49] employed a learning-free energy belief propagation scheme to propagate energy values across the input graph, thereby ensuring the distinguishability between ID and OOD nodes. For group-level methods, GOOD-D [8] proposed a hierarchical contrastive learning framework to explore the discrepancies between ID and OOD graphs at various levels, including node-level, graph-level, and group-level. GraphDE [9] characterized the distribution shifts by modeling the generative process. It employed a recognition model and structure estimation model to model and infer the latent environment variable for OOD detection.

Another related research direction is graph anomaly detection. There are many GNN-based methods for node or edge anomaly detection [50], [51], [52], [53]. In our experiments, we also compare with OCGIN [54] and GLocalKD [55], two SOTA methods for detecting anomalous graphs.

In contrast to these methods, our DGP approach can both take advantage of pre-trained GNN encoders, and utilize class label information to capture finer-grained ID patterns. Table I shows the relationship of relevant OOD detection work. Here *fine-grained modeling* indicates that a method considers ID patterns from different aspects.

III. PRELIMINARY

A. Notations and Problem Formulation

1) *Notations*: Given two distinct distributions in graph space denoted as $\mathcal{P}_{ID}$ and $\mathcal{P}_{OOD}$, we only have access to graphs

from $\mathcal{P}_{ID}$ during the training phase. Let the training dataset $\mathcal{D}_{ID} = \{(G^k, y^k)\}_{k=1}^N$ represent N graphs sampled from in-distribution $\mathcal{P}_{ID}$, where $\mathcal{Y}$ is the label set of ID graphs, and $y^k \in \mathcal{Y}$. Specifically, each graph in this dataset is denoted as $G^k = \{\mathcal{V}^k, \mathcal{E}^k, X^k\}$, where $\mathcal{V}^k$ is the set of nodes and $\mathcal{E}^k \subseteq \mathcal{V}^k \times \mathcal{V}^k$ is the set of edges. $X^k = [x_1^k, x_2^k, \dots, x_{|\mathcal{V}^k|}^k] \in \mathbb{R}^{|\mathcal{V}^k| \times d}$ denotes the node feature matrix, where x_i^k is a d -dimensional feature vector of node $v_i^k \in \mathcal{V}^k$. Besides, $A^k \in \{0, 1\}^{|\mathcal{V}^k| \times |\mathcal{V}^k|}$ represents the adjacency matrix of graph G^k , where $a_{ij}^k = 1$ when there is an edge e_{ij}^k connecting vertex v_i^k and v_j^k in G^k .

2) *Problem Definition*: OOD detection can be formulated as a binary classification problem, whose goal is to decide whether a graph G^t is from $\mathcal{P}_{ID}$ or not during the test phase. This decision process can be written as follows:

$$\text{decision} = \begin{cases} \text{ID}, & \text{if } S(G^t) \geq \delta \\ \text{OOD}, & \text{if } S(G^t) < \delta \end{cases}, \quad (1)$$

where the scoring function $S(\cdot, \cdot)$ takes the adjacency matrix A^t and node feature matrix X^t as inputs, and is supposed to assign larger values to ID graphs. Most evaluations employ ranking-based metrics such as AUC, and thus we do not need to specify the threshold δ .

B. A Typical Framework for OOD Detection

Due to the unavailability of OOD data during the training phase, directly applying classifier-based OOD detection methods to graph encoders becomes challenging. In this paper, we consider a typical two-step framework to implement the scoring function $S(\cdot, \cdot)$. The first step is to encode each graph G^t into vector representation h^t by a GNN. Then in the second step, a mapping function will project the graph representation to a scalar as the decision score.

Formally, we denote the GNN encoder as $f(A^t, X^t; \Theta)$ with parameters Θ , and compute the detection score by

$$S(G^t) = \text{MD}(f(A^t, X^t; \Theta)), \quad (2)$$

where $\text{MD}(\cdot)$ is a non-parametric mapping function based on the Mahalanobis distance. Briefly, Mahalanobis Distance employs K-means clustering to partition the representations into Q clusters, and calculates the score of h^t by its distance from the corresponding cluster center:

$$\text{MD}(h^t) = \{\min_q (h^t - \hat{\mu}_q)^\top \hat{\Sigma}_q^{-1} (h^t - \hat{\mu}_q)\}^{-1}, \quad (3)$$

where $\hat{\mu}_q$ and $\hat{\Sigma}_q$ represent the mean and covariance of representations within cluster q . The Mahalanobis distance has been extensively used in OOD detection [10], [56], [57], and $Q = 1$ is a typical setting in practice.

IV. THE PROPOSED MODEL

In this section, we present the proposed model DGP that aims to characterize fine-grained ID patterns under the *pre-training+prompting* paradigm for graph OOD detection. We start with the overview of DGP. Our approach involves jointly reasoning with both class-specific and class-agnostic information for OOD detection. Inspired by prompt learning, we design two graph generators to extract the aforementioned two patterns. Lastly, we discuss the optimization process and advantages, and time complexity of our proposed model.

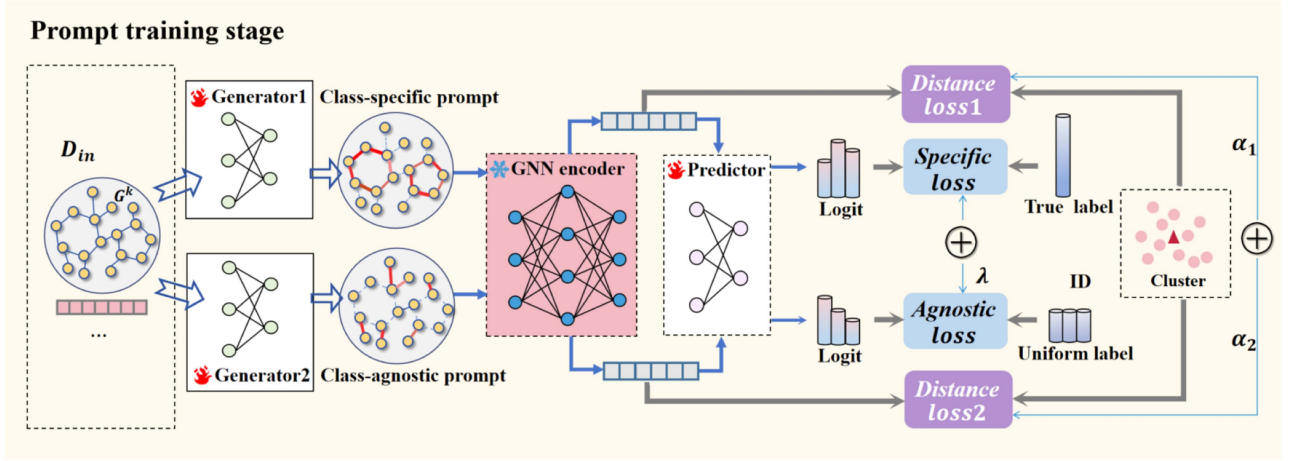


Fig. 2. An illustration of the training process of our proposed DGP. For each input graph, we generate two prompt graphs to capture ID patterns from class-specific and class-agnostic views, respectively. We design several losses to supervise and regularize the training of prompt generators. Here the parameters of pre-trained GNN encoder are frozen in our method.

A. Overall Framework

The core idea of our work is to recognize both class-specific and class-agnostic patterns from ID graphs. Both types of patterns are supposed to contain useful clues for distinguishing OOD graphs from ID ones, and thus enable graph OOD detection in a finer-grained view.

The overall framework is shown in Fig. 2. Two separate generators construct class-specific and class-agnostic prompt graphs by modifying the edge weights of the original graph. Then the pre-trained GNN encoder projects prompt graphs into vector representations, and the predictor further converts the representations into class distributions. During the training phase, class-specific and class-agnostic losses are employed to supervise the learning of two generators. The distance losses encourage the prompt graphs to be different from the original graph, and can be seen as regularization terms preventing trivial solutions. The generated prompt graphs can highlight two key patterns of ID graphs, thereby facilitating the distinction between ID and OOD graphs. During the test phase, we define the decision score function based on the encoded representations of two prompt graphs and the aforementioned Mahalanobis distance:

$$S(G^t) = \text{MD}(f(\phi_1(G^t; \Omega_1), X^t; \Theta)) + \gamma \text{MD}(f(\phi_2(G^t; \Omega_2), X^t; \Theta)), \quad (4)$$

where ϕ_1 and ϕ_2 are respectively the class-specific and class-agnostic prompt generators, γ is a hyper-parameter balancing the two parts.

In this way, the pre-trained GNN model f is prompted to fit the OOD detection task without retraining. In the following subsections, we will provide a detailed introduction to the prompt graph generation and the training strategy for OOD detection.

B. Prompt Graph Generator

To bridge the gap between the pre-training task and the downstream OOD detection task, we draw inspiration from prompting in natural language processing. Since manually creating prompts requires a significant amount of time and expertise, we will learn

prompt generation functions to design prompts for every input graph.

Considering that only in-distribution data is visible during the training phase for graph OOD detection, an intuitive approach for generating graph prompts is to amplify the key patterns within the in-distribution data, thereby increasing the difference between in-distribution and out-of-distribution graphs. Specifically, we will modify the topology structure which reflects the characteristics of a graph.

Firstly, we formalize the pre-trained GNN model f by a three-step form:

$$\begin{aligned} a_i^{k,l} &= \text{UPDATE}^{(l)}(a_i^{k,l-1}, \text{AGG}^{(l)}(\{a_j^{k,l-1} \mid (v_i^k, v_j^k) \in \mathcal{E}^k\})), \\ a_i^k &= \text{CONCAT}(\{a_i^{k,l} \mid l = 1, 2, \dots\}), \\ h^k &= \text{PROJ}(\text{READOUT}(\{a_i^k \mid v_i^k \in \mathcal{V}^k\})), \end{aligned} \quad (5)$$

where $a_i^{k,l}$ is the representation of node v_i^k in G^k at the l -th layer, and $a_i^{k,0} = x_i^k$. $\text{UPDATE}(\cdot)$ and $\text{AGG}(\cdot)$ are message passing and aggregation operations, respectively. We concatenate the node representations in each layer using the $\text{CONCAT}(\cdot)$ operation. Additionally, we employ the pooling operation $\text{READOUT}(\cdot)$ to transform the node representations into a holistic graph representation. Moreover, a non-linear transformation function PROJ is introduced to encode information, leading to the final representation h^k .

Given an input graph G^k , we create graph prompts by adjusting the topological structure of the input graph as prompts. This involves using a trainable graph adjacency matrix $A^{k,*}$ for graph OOD detection.

Specifically, we compute edge weights based on node features to highlight key ID patterns. Note that encoded node representations contain more meaningful information compared to the initial attributes. Thus, we make full use of the pre-trained GNN encoder, and calculate the edge weight between node v_i^k and v_j^k in G^k as

$$A_{ij}^{k,*} = \text{MLP}(\text{CONCAT}(a_i^k, a_j^k); \Omega), \quad (6)$$

where a_i^k and a_j^k are node representations in (5). Here we concatenate the representations of node v_i^k and v_j^k , and employ a multi-layer perceptron $\text{MLP}(\cdot)$ parameterized by Ω for simplicity. Also, we only adjust the weight of existing edges to enjoy a linear computational complexity.

For class-specific and class-agnostic sides, we use two independent MLPs with parameters Ω_1 and Ω_2 . Then we correspondingly have two prompt graphs as $A^{k,1} = \phi_1(G^k; \Omega_1)$ and $A^{k,2} = \phi_2(G^k; \Omega_2)$.

C. Encouraging Disentanglement

The quality of generated prompts has a significant impact on downstream tasks. Since OOD data is not available during the training process, we can not optimize (4) to train prompt generators $\phi_1(G^t; \Omega_1)$ and $\phi_2(G^t; \Omega_2)$. In this work, we use label information of ID graphs to encourage the two generators to respectively capture ID patterns from class-specific and class-agnostic spaces.

1) *Class-Specific Part*: As OOD graphs may vary greatly from each class of ID graphs, the information related to the label of ID samples is important for OOD detection. The prompt generator ϕ_1 is encouraged to identify the key edges of ID graphs related to classification.

Formally, we use the pre-trained GNN encoder to transform prompt graph $A^{k,1}$ into representation $h^{k,1}$, and further employ a two-layer $\text{MLP}(\cdot)$ as a $|\mathcal{Y}|$ -class label predictor to estimate the class distribution of the prompt graph:

$$h^{k,1} = f(A^{k,1}, X^k; \Theta), \quad z^{k,1} = \text{softmax}(\text{MLP}(h^{k,1}; \Psi)), \quad (7)$$

where Ψ denotes the parameters in the predictor.

Then we align the predicted label distribution $z^{k,1}$ with the ground truth label y^k of G^k as the class-specific loss:

$$\mathcal{L}_{\text{class-specific}} = \frac{1}{|\mathcal{D}_{\text{ID}}|} \sum_{G^k \in \mathcal{D}_{\text{ID}}} \text{CE}(y^k, \log(z^{k,1})), \quad (8)$$

where CE is the cross entropy function.

The idea of this part shares some merits with causal subgraph discovery works [58], [59], [60]. But they aim to train interpretable and generalizable GNNs, and our goal is to prompt a well-trained GNN for OOD detection task without adjusting its parameters. The implementation details and other modules of our method are also quite different.

2) *Class-Agnostic Part*: Although class-agnostic patterns are not essential for ID graph classification, they may contain useful information related to ID/OOD identification. The class-agnostic pattern captures shared information among the ID graphs, providing an opportunity to formulate additional constraints for the ID data. Therefore, we build another prompt from a complementary view. We use the same GNN encoder and label predictor as the class-specific part. But we expect the predicted label distribution to align uniform distribution $\bar{y}$ instead.

$$h^{k,2} = f(A^{k,2}, X^k; \Theta), \quad z^{k,2} = \text{softmax}(\text{MLP}(h^{k,2}; \Psi)), \quad (9)$$

$$\mathcal{L}_{\text{class-agnostic}} = \frac{1}{|\mathcal{D}_{\text{ID}}|} \sum_{G^k \in \mathcal{D}_{\text{ID}}} \text{CE}(\bar{y}, z^{k,2}). \quad (10)$$

In this way, the class-agnostic prompt is encouraged to capture some small fingerprints that appear in all classes of ID graphs.

Finally, to incorporate information from both class-specific and class-agnostic ID patterns, the objective of disentanglement loss can be defined as the weighted sum of the losses:

$$\mathcal{L}_{\text{disentangle}} = \mathcal{L}_{\text{class-specific}} + \lambda \mathcal{L}_{\text{class-agnostic}}, \quad (11)$$

where λ is a hyper-parameter.

3) *Distance Loss as Regularizations*: Directly optimizing (11) sometimes leads to trivial solutions, e.g., learned class-specific prompts assign large weights to all edges in the input graphs. Inspired by the minimal sufficient principle used in information bottleneck [61], there is an encouragement to incorporate as much information related to OOD detection as possible into the representation to make the predictions as comprehensive as possible, while simultaneously preventing the representation from including additional information unrelated. Thus, we introduce the following distance losses to push both class-specific and class-agnostic prompts away from the original graphs:

$$\mathcal{L}_{\text{distance-1}} = \frac{1}{|\mathcal{D}_{\text{ID}}|} \sum_{G^k \in \mathcal{D}_{\text{ID}}} \text{MD}(h^{k,1}), \quad (12)$$

$$\mathcal{L}_{\text{distance-2}} = \frac{1}{|\mathcal{D}_{\text{ID}}|} \sum_{G^k \in \mathcal{D}_{\text{ID}}} \text{MD}(h^{k,2}), \quad (13)$$

$$\mathcal{L}_{\text{dis}} = \alpha_1 / \mathcal{L}_{\text{distance-1}} + \alpha_2 / \mathcal{L}_{\text{distance-2}}, \quad (14)$$

where α_1 and α_2 are hyper-parameters. This loss can help the two generators keep the most informative and representative edges, and improve the performance at test stage.

D. Discussion

1) *Details About Training Stage*: We design an iterative process to train the model parameters, including Ω_1 in the class-specific prompt generator ϕ_1 , Ω_2 in the class-agnostic prompt generator ϕ_2 , and Ψ in the classification predictor. Firstly, we update Ω_1, Ω_2 in the prompt generation functions, and Ψ by minimizing (11); Secondly, we fix Ψ , and update Ω_1, Ω_2 according to (14). The pseudo-code of DGP is presented in Algorithm 1.

2) *Details About Testing Stage*: In the testing stage, we drop the predictor and compute the decision score of each test graph G^t by (4). Graphs with higher/lower scores are recognized as ID/OOD ones.

3) *Benefits of DGP*: The advantages of our method are three-fold: (1) Our proposed DGP leverages the inherent capabilities of well-trained GNNs, enabling them with the ability for OOD detection. (2) DGP can effectively reuse a pre-trained GNN encoder without adjusting its parameters, showing the great potential of the *pre-training+prompting* paradigm in this task. (3) The framework of DGP is versatile, compatible with the classical scoring function, as it does not make any assumptions about it. (4) DGP considers ID patterns from both class-specific and class-agnostic perspectives, enabling a finer-grained characterization of ID graphs. (5) The prompt graphs generated by DGP can offer some interpretability to the decisions, which will be investigated by case study in our experiments.

4) *Time Complexity Analysis*: We further analyze the time complexity of the proposed DGP. Let the training set contain N ID graphs. For a single graph with n nodes, m edges, the per-iteration time complexity consists of the following components:

(1) **Node representation extraction**. Node representations are extracted using a pre-trained GNN. For an L -layer GNN, each layer costs $O(md)$ for message aggregation and $O(nd^2)$

Algorithm 1: Disentangled Graph Prompting (DGP)

Input: Training set of ID graphs $\mathcal{D}_{ID}$, pre-trained GNN model f with parameters Θ , random initialized MLP with parameters Ψ for prediction;
Parameter: Randomly initialize Ω_1 and Ω_2 for prompt generator ϕ_1 and ϕ_2 , respectively;
Output: Prompt generator ϕ_1 with learned parameters Ω_1 , prompt generator ϕ_2 with learned parameters Ω_2 ;
1: **while** not converge **do**
2: **for** each graph $G^k \in \mathcal{D}_{ID}$ **do**
3: Compute a_i^k for each node v_i^k by (5);
4: Compute the prompt graph representation $h^{k,1}$ and the predicted label distribution $z^{k,1}$ by (7) for class-specific part;
5: Compute the prompt graph representation $h^{k,2}$ and the predicted label distribution $z^{k,2}$ by (9) for class-agnostic side;
6: **end for**
7: Update Ω_1 , Ω_2 and Ψ by minimizing (11);
8: Update Ω_1 and Ω_2 by minimizing (14);
9: **end while**
10: **return** Learned parameters Ω_1 and Ω_2 for prompt generator ϕ_1 and ϕ_2 , respectively.

for node updates. Thus, the total cost per graph is $O(L(md + nd^2))$. **(2) Prompt graph generator computation.** Two prompt graph generators ϕ_1 and ϕ_2 compute edge weights using MLPs that take $2d$ -dimensional concatenated node embeddings as input. For m edges and hidden dimension d , the complexity is $O(2md^2)$. **(3) GNN inference on prompt graphs.** Both prompt graphs are encoded by the GNN, requiring two forward passes, each with complexity $O(L(md + nd^2))$, totaling $O(2L(md + nd^2))$ per graph. **(4) Classification prediction.** A lightweight MLP maps the d -dimensional graph representation to C class logits with cost $O(dC)$.

Combining all steps, the total per-iteration complexity over the N ID graphs is:

$$T_{\text{iter}} = \mathcal{O}(N \cdot [3L(md + nd^2) + 2md^2 + dC]), \quad (15)$$

simplifying, the dominant terms yield:

$$T_{\text{iter}} = \mathcal{O}(Nd \cdot [L(m + nd) + md + 1]). \quad (16)$$

This analysis demonstrates that DGP maintains manageable computational complexity and is scalable to large datasets.

V. EXPERIMENTS

In this section, we evaluate the effectiveness of DGP on graph OOD detection task. Then we conduct ablation studies to verify the method design of DGP, analyze the importance of pre-trained GNN encoders. We also explore hyper-parameter sensitivity, and investigate the effect of MLP layers in the prompt graph generator. Furthermore, we analyze the training efficiency of DGP and visualize the generated class-specific and class-agnostic prompt graphs.

A. Experimental Setup

1) *Datasets:* We conduct experiments on ten pairs of ID and OOD datasets following the experimental settings in GOOD-D [8] and AAGOD [10]. These datasets are ten widely-used

datasets for graph OOD detection from TU datasets [62] and OGB datasets [63], covering diverse domains, such as molecular datasets, social network datasets, and bioinformatics datasets. These datasets exhibit variations and differences in their distributions. Following the experimental setup described in [8], we train our DGP model with 80% of ID samples as the training set. For the validation/test set, we combine 10% ID samples and the same number of OOD ones. Specially, for social networks, node labels are used as input features. Since the input of the social networks has neither node nor edge labels, node degrees are used as the input features.

2) *Baselines:* We compare our DGP with baselines in the following four categories:

- *Non-Graph-Based Methods:* We include five representative non-graph OOD detection approaches: NegLabel [64], AdaNeg [65], Local-Prompt [66], PFSOOD [67], and PRO [68]. These methods, originally designed for vision-language or feature-based OOD detection, are adapted to the graph domain for fair comparison.
- *Pre-Training-Based Methods:* We compare our DGP with pre-trained GNNs (GCL [14], SimGRACE [15]) to see whether DGP can improve their OOD detection performance. To further demonstrate the superiority of prompting, we also compare with their fine-tuned versions (GCL-ft, SimGRACE-ft), where two identically initialized GNN encoders are updated according to the losses used by DGP.
- *Graph Anomaly Detection Methods:* We include two classical graph anomaly detection baselines: OCGIN [54] and GLocalKD [55], which are optimized for identifying graph-level anomalies through one-class learning or knowledge distillation.
- *Graph-Based Methods:* We further compare with several SOTA graph OOD detection models, including GraphDE [9], GOOD-D [8], AAGOD [10], GOODAT [13], HGOE [12], and SEGO [11]. These methods represent the current leading designs for graph-level OOD detection based on distributional modeling, contrastive learning, or adaptive prompting.

3) *Selections of Pre-Trained GNNs:* Since our DGP is compatible with any well-trained Graph Neural Networks (GNNs). In this paper, to comprehensively assess DGP's efficacy, we select two self-supervised methods to pre-train the GNN encoder: GCL [14] and SimGRACE [15]. Both GCL and SimGRACE are based on contrastive learning [69], with a three-layer GIN as the encoder and a two-layer MLP as the projection head. Their difference lies in the way they generate augmentation views. GCL and SimGRACE employ distinct augmentation strategies, with the former focusing on data-level augmentation and the latter at the embedding-level. Therefore, utilizing these two self-supervised approaches to validate the effectiveness of our method establishes a compelling and comprehensive validation strategy. More details about pre-training methods can be found in the following:

- **GCL [14]** employs four data augmentation strategies to generate augmentation views, including node dropping, edge perturbation, attribute masking, and subgraph sampling.
- **SimGRACE [15]** adds slight perturbations to the model parameters of graph encoder to construct augmentation views. The weight of perturbation is a crucial hyper-parameter in SimGRACE [15], and we adjust it in the range of $\{0.1, 1.0, 10.0, 100.0, 1000.0\}$ as [15] did.

TABLE II

OOD DETECTION RESULTS IN TERMS OF AUC(%). THE BEST RESULTS ARE HIGHLIGHTED IN BOLDFACE, AND THE SECOND-BEST RESULTS ARE UNDERLINED.

ID-dataset	BZR	PTC_MR	AIDS	ENZYMES	IMDB-M	Tox21	FreeSolv	BBBP	ClinTox	Esol	Avg.
OOD-dataset	COX2	MUTAG	DHFR	PROTEIN	IMDB-B	SIDER	ToxCast	BACE	LIPO	MUV	
NegLabel	41.18	56.61	34.79	54.53	58.38	51.65	45.59	46.85	44.97	48.81	48.34
AdaNeg	47.69	54.06	72.15	58.90	49.72	43.25	49.82	42.42	55.68	27.08	50.08
Local-Prompt	73.09	59.90	68.61	43.09	76.14	25.00	25.20	44.30	43.30	10.00	46.86
PFSOOD	64.08	61.57	10.06	50.84	72.29	50.45	69.76	46.01	42.49	47.12	51.47
PRO	52.05	40.86	86.64	55.71	37.52	46.00	36.00	44.00	57.00	24.00	47.98
GCL	83.56	62.89	97.54	68.97	76.65	68.17	80.73	71.11	57.64	80.02	74.73
GCL-ft	77.38	84.26	97.75	69.47	82.33	66.80	81.49	75.85	58.92	81.91	77.62
SimGRACE	92.50	65.40	97.08	61.11	65.18	67.38	75.22	59.43	58.84	77.87	72.00
SimGRACE-ft	87.38	79.67	97.93	61.83	80.79	67.26	83.01	76.02	54.96	79.29	76.81
OCGIN	76.66	80.38	86.01	57.65	67.93	46.09	59.60	61.21	49.13	54.04	63.87
GlocalKD	75.75	70.63	93.67	57.18	78.25	66.28	64.82	73.15	55.71	86.83	72.23
GraphDE	82.09	81.31	62.34	62.79	81.99	67.98	58.79	66.22	53.30	73.06	68.99
GOOD-D	94.99	81.21	99.07	61.84	79.94	66.50	80.13	82.91	69.18	91.52	80.73
AAGOD-GCL	97.31	65.74	98.03	73.76	83.84	71.27	71.51	80.64	69.29	78.17	78.96
AAGOD-Sim	94.19	69.03	93.27	72.58	81.38	67.19	69.92	78.33	55.90	76.45	75.82
GOODAT	82.16	81.84	96.43	66.29	79.03	68.92	68.83	77.07	62.46	85.91	76.89
HGOE	80.96	75.10	<u>99.28</u>	64.44	81.74	68.24	82.89	<u>83.46</u>	70.09	92.64	79.88
SEGO	96.66	<u>85.02</u>	99.48	64.42	80.27	66.67	90.95	87.55	78.99	94.59	84.46
DGP-GCL	<u>98.25</u>	88.24	97.34	75.22	<u>85.05</u>	86.04	88.67	81.08	80.58	89.49	87.00
DGP-Sim	99.12	84.08	98.99	<u>74.31</u>	85.60	<u>76.37</u>	91.72	80.70	78.76	95.13	<u>86.48</u>

We employ the same settings as previous GCL methods [14], [15], [70], [71]. For the graph encoder, we adopt a 3-layer GIN [72] with 32 hidden and output dimensions. For the projection head, we use a two-layer MLP with 96 hidden and output dimensions. We employ Adam optimizer [73] to update all the parameters, and the batch size is set to 128. The learning rate of GIN is tuned in $\{0.1, 0.01, 0.001\}$ as previous works.

4) *Evaluation Metrics*: We follow existing works [74], [75] and use three evaluation metrics [76] to measure the performance of OOD detection models.

AUC $\uparrow$ stands for the area under the ROC curve. The closer the curve is to the upper-left corner, the better the overall performance of the model. A higher AUC score indicates better OOD detection performance of the model.

AUPR $\uparrow$ is the area under the PR curve, where PR stands for the curve composed of recall rate and precision. A higher AUPR value indicates better OOD detection performance.

FPR95 $\downarrow$ quantifies the rate of misclassifying ID samples as OOD when the true positive rate reaches 95%. A lower FPR95 value indicates better performance, as it reflects a lower ratio of false positives.

5) *Implementation*: We run all our experiments on a single GPU device of GeForce RTX 3090 with 24 GB memory. Besides, we implement our framework based on Python of version 3.9.0, PyTorch Geometric (PyG) of version 2.0.4 and PyTorch of version 1.11.0.

In our proposed DGP, we simply use a two-layer MLP as the prompt graph generator for generating prompt graphs. The hidden layer dimension of MLP is 32. We adjust the learning rate of Adam optimizer within $[10^{-4}, 10^{-1}]$, the values of λ, γ within the range of $\{0.1, 0.5, 1, 5, 10\}$, and the values of α_1, α_2 within the range of $\{10^2, 10^3, 10^4, 10^5\}$.

B. Experimental Evaluation

1) *Comparison With Non-Graph-Based Methods*: We first compare our DGP with several non-graph OOD detection approaches adapted to the graph domain. All methods are evaluated under the same benchmark setting, and the results are reported in Table II.

As shown in the results, non-graph methods perform significantly worse than graph-based approaches. Their average AUC is only 48.95%, far below that of DGP and other graph OOD baselines. This indicates that directly transferring non-graph paradigms to graph data is ineffective, as these methods fail to capture the structural dependencies and relational information that are critical for graph OOD detection.

2) *Comparison With Methods Based on Pre-Training*: In this experiment, we explore whether DGP yields performance improvements compared to pre-trained GNNs and their fine-tuned versions for graph OOD detection. The results are presented in Table II with GCL [14] or SimGRACE [15] to pre-train GNN encoders. GCL represents directly use the pre-trained GNN encoder for graph OOD detection. GCL-ft removes the graph prompt generators from both branches and fine-tuned the pre-trained GNN encoder using two different Adam optimizers under the supervision of disentangle loss and distance loss.

Based on results, we make the following observations:

- Compared with pre-trained GNN/fine-tuned GNN/AAGOD, our proposed DGP has 19.86%/13.65%/13.39% relative AUC improvement on average, and thus achieves the best overall performance.
- Prompt-based methods (AAGOD and DGP) outperform the fine-tuned GNNs, confirming the superiority of *pre-training+prompting* over *pre-training+fine-tuning* on this problem.

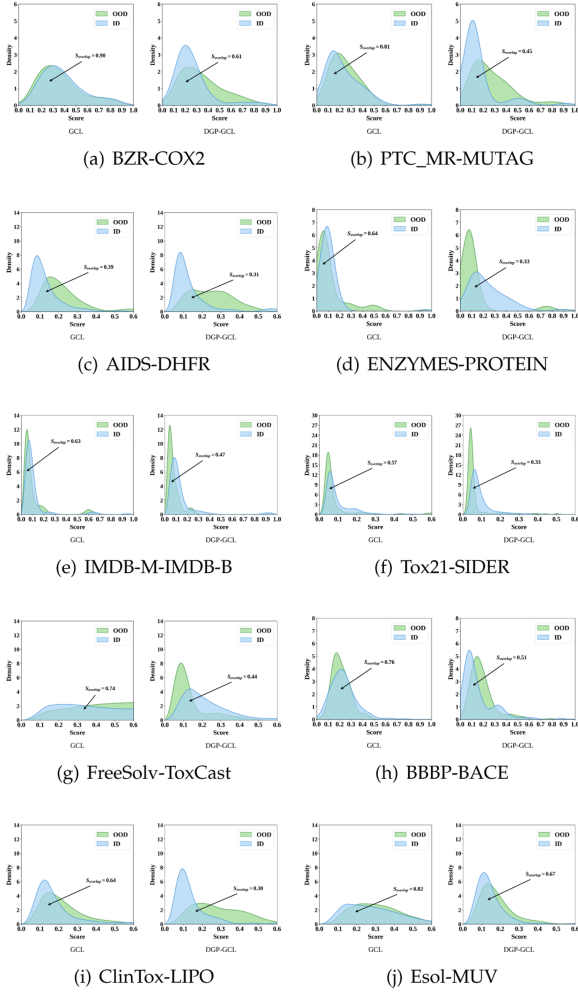


Fig. 3. Decision score distributions of ID and OOD graphs on ten datasets. Smaller distribution overlap indicates better detection ability. The two images in the same column respectively represent the pre-trained (left), pre-trained+prompted right.

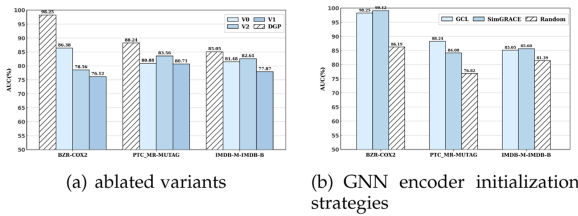


Fig. 4. Graph OOD detection results (AUC values) of different ablated variants and GNN encoder initialization strategies.

- Compared with AAGOD, DGP greatly improves the graph OOD detection performance on multiple datasets. One possible reason is that the feature space learned by AAGOD for graph OOD detection is not informative. The improvement over AAGOD shows the effectiveness of our fine-grained disentanglement modeling.
- In different datasets, DGP-GCL and DGP-SimGRACE alternatively attain the best performance, suggesting that different pre-trained models may be well-suited for distinct datasets. They both outperform the baseline models, further confirming the usability of our DGP.

3) *Comparison With Other SOTA Methods:* To validate our motivation that using fine-grained modeling for detecting OOD graphs is feasible, we also compare our DGP with various methods on graph OOD detection. The comparison against these SOTA methods for the ten pairs of datasets is shown in Table II, we draw the following findings:

- Our DGP achieves SOTA performance on 8 out of 10 datasets, and on average has 3.63% relative improvement over SEGO, the best baseline. In other words, our DGP can achieve better results without the need to retrain any GNN encoders.
- Compared with GraphDE [9] that also utilizes label information of ID graphs, DGP can benefit from high-quality pre-trained encoders to better represent graph structures.
- In addition, graph anomaly detection methods have relatively poor performance on this task. As the methods specifically designed for the graph anomaly detection task struggle to handle the OOD detection task effectively, we should treat the OOD detection problem differently from conventional anomaly detection.

4) *Visualization of Score Gap:* We visualize the distributions of decision scores on ten datasets in Fig. 3. For pre-trained GNN, the average distribution overlap is 0.69, while for DGP, the average distribution overlap is 0.44. Compared with pre-trained GNN, our proposed DGP has a 35.94% relative distribution overlap reduction on average. We can observe that the decision scores with the representations from pre-trained encoders are highly concentrated. After the prompting of DGP, the scores of OOD samples become more dispersed in the distribution of decision scores. This indicates that OOD samples, due to the absence of the two types of ID patterns, can be better distinguished by DGP. Following fine-grained modeling, the disparity between ID and OOD data is widened, showcasing the enhanced capability of the pre-trained model in graph OOD detection.

C. Model Analysis

1) *Ablation Study:* Since fine-grained modeling involving disentangle loss and distance loss in DGP guarantees the disentanglement of ID patterns, we conduct an ablation study to validate their effectiveness. In particular, we compare with the following ablated variants on three datasets:

- **V0** directly removes the branch for mining class-agnostic patterns, and retains only the prompt graph generator and distance loss for class-specific patterns to verify the impact of class-specific ID patterns on graph OOD detection.
- **V1** directly removes the branch for mining class-specific patterns, and retain only the prompt graph generator and distance loss for class-agnostic patterns to verify the impact of class-agnostic ID patterns on graph OOD detection, similar to V0.
- **V2** removes the distance loss and retains only the disentangle loss for training prompt graph generators designed for class-specific and class-agnostic ID patterns.

As shown in Fig. 4(a), our full model DGP always has better results than other ablated variants, demonstrating the necessity of each module in DGP. V0 and V1 have competitive performance. Both the class-specific and class-agnostic branches are complementary to each other, contributing to further performance improvement, which validates our motivation of fine-grained disentanglement. The regularization loss that

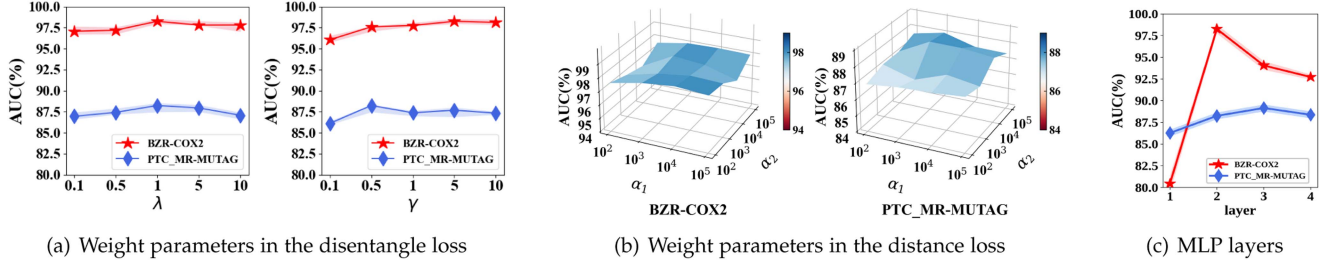


Fig. 5. Graph OOD detection results (AUC values) on BZR-COX2 and PTC_MR-MUTAG with respect to four hyper-parameters and different MLP layers.

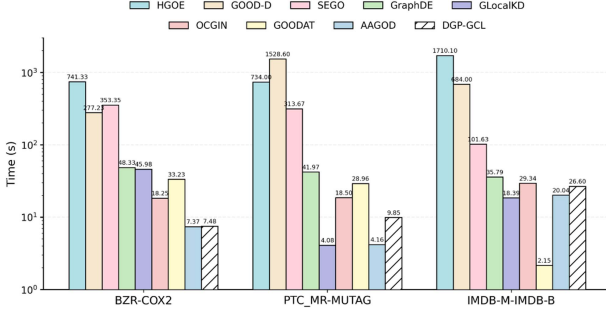


Fig. 6. Running time compared with other SOTA models.

helps prevent trivial solutions also plays a vital role to the training of DGP.

2) *Effect of Pre-Trained GNN Encoders*: The DGP framework employs pre-trained GNN encoders without requiring retraining. To validate the effectiveness of this design, we compare the AUC performance of DGP under different GNN encoder initialization strategies (GCL pre-training, SimGRACE pre-training, and random initialization), as shown in Fig. 4(b). Results demonstrate that pre-trained encoders consistently and substantially outperform randomly initialized counterparts. Specifically, random initialization yields markedly lower AUC scores across all datasets. For instance, on BZR-COX2, PTC_MR-MUTAG, and IMDB-M-IMDB-B, random initialization yields AUC scores that are, on average, 11.12% lower than GCL and 9.88% lower than SimGRACE, respectively. This suggests that without pre-training, DGP fails to effectively capture fine-grained structural patterns distinguishing ID from OOD graphs, leading to degraded detection performance. These findings conclusively validate the necessity of integrating pre-trained GNNs into the DGP framework.

3) *Hyper-Parameter Sensitivity*: In this section, we investigate the sensitivity of hyper-parameters and report the results of graph OOD detection (AUC) on BZR-COX2 and PTC_MR-MUTAG with validation set in Fig. 5. We can see that DGP has generally stable performance, and is robust as the hyper-parameters change.

- *Value of weight parameters in the disentangle loss*: λ and γ are key parameters controlling the importance of class-specific and class-agnostic patterns during the training and testing phases, respectively. Therefore, we vary λ and γ in $\{0.1, 0.5, 1, 5, 10\}$, and test the performance of DGP. As shown in Fig. 5(a), for the BZR-COX2 dataset, class-agnostic patterns are relatively more important, while for the PTC_MR-MUTAG dataset, class-specific patterns are relatively more important. The above observations confirm

the necessity of addressing graph OOD detection through fine-grained modeling.

- *Value of weight parameters in the distance loss*: Considering the influence of weight parameters α_1 and α_2 in the distance loss during the training phase, we modify their values and evaluate how they impact the graph OOD detection performance. We vary α_1, α_2 in $\{10^2, 10^3, 10^4, 10^5\}$ as shown in Fig. 5(b), with the increase in the values of weight parameters α_1 and α_2 , the performance of DGP initially rises and then declines. One possible reason is that smaller weight parameters may not provide sufficient capacity to avoid trivial solutions, while larger weight parameters may not offer enough attention to fine-grained disentanglement.

4) *Effect of MLP Layers*: We investigate the effect of varying the number of layers in the MLP used by the prompt graph generator. As shown in Fig. 5(c), the performance impact differs across datasets. on BZR-COX2, increasing layers from 1 to 2 improves AUC substantially, peaking at 2 layers. Further depth causes minor performance declines, implying deeper architectures risk overfitting or optimization instability. Conversely, performance on PTC_MR-MUTAG remains stable across 2–4 layers, indicating diminishing returns from increased depth. Overall, a 2-layer MLP achieves optimal balance between expressiveness and generalization, delivering peak performance on BZR-COX2 and competitive results on PTC_MR-MUTAG. We thus adopt this configuration as the default to maintain efficiency without sacrificing effectiveness.

5) *Efficiency Analysis*: To further validate the efficiency of our DGP, we compare the training time of DGP-GCL with representative baselines that exhibit strong AUC performance, as shown in Fig. 6.

As shown in the results, DGP-GCL consistently achieves substantial reductions in training time. On BZR-COX2, DGP-GCL outperforms SEGO and GOOD-D in both performance and efficiency, reducing training time from 353.35 s (SEGO) and 277.23 s (GOOD-D) to just 7.48s—a 46× to 36× speedup. On PTC_MR-MUTAG, DGP-GCL achieves over 31× faster training compared to SEGO (313.67 s vs. 9.85 s) and also runs significantly faster than GOODAT (28.96 s). The most substantial improvement occurs on IMDB-M-IMDB-B, where DGP-GCL demonstrates a 64× reduction in training time compared to HGOE (1710.10 s vs. 26.60 s) and a modest 1.3× improvement over GraphDE (35.79 s vs. 26.60 s). These consistent time savings are achieved without compromising detection performance, as the selected baselines represent the top-performing models in terms of AUC.

The above results demonstrate the key advantage of DGP: by reusing a pretrained GNN encoder and avoiding redundant

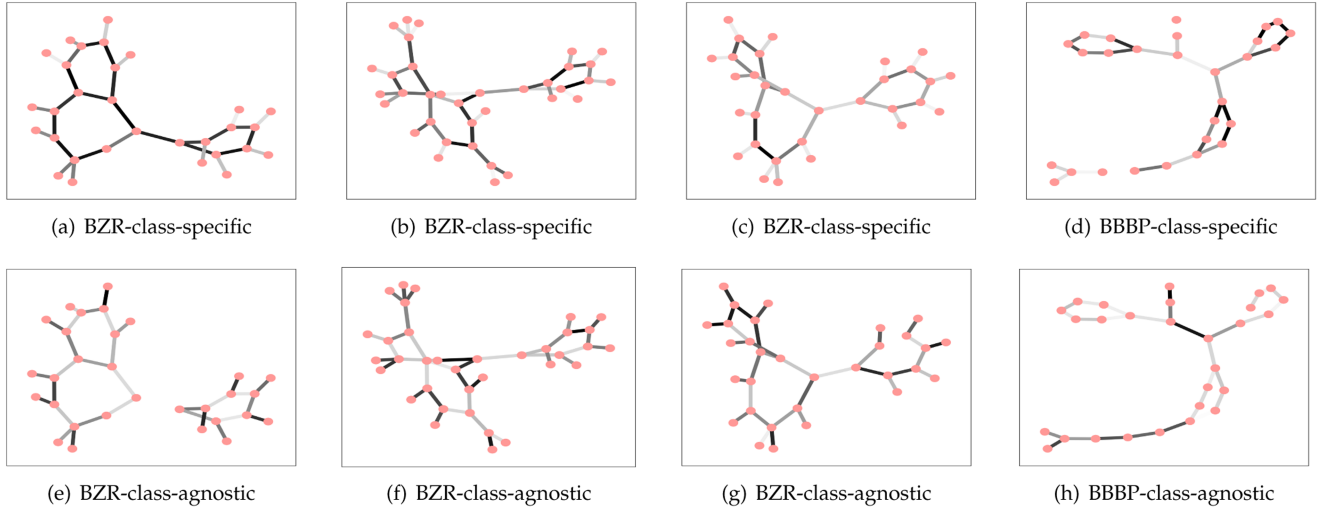


Fig. 7. Visualizations of two types of prompt graphs as case study analysis. Here darker edges indicate larger learned weights. The two images in the same column respectively represent the class-specific prompt graph (top) and the class-agnostic prompt graph (bottom).

end-to-end training, our method greatly improves computational efficiency.

6) *Visualization of Prompt Graphs*: To gain further insights into the learned prompt graphs, here we conduct case studies by visualizing generated prompt graphs on BZR and BBBP datasets in Fig. 7. It is worth noting that the edges in the amplified graphs are directional, and we calculate the average weights of identical edges in different directions for simplicity. As shown in Fig. 7(a), 7(b), 7(c) and 7(d), the class-specific prompts capture key substructures in ID graphs, such as the backbone structure. While in Fig. 7(e), 7(f), 7(g) and 7(h), class-agnostic prompts focus on the edges that are unlikely to be related to ID class labels. The two types of prompts can complement with each other to provide richer ID patterns, aligning with the results in the ablation study. In summary, the learned prompt graphs effectively emphasize crucial positions that contribute to graph OOD detection, thereby accentuating the distinctions between ID and OOD graphs.

VI. CONCLUSION

In this paper, we innovatively propose to discern fine-grained ID patterns for OOD detection under the paradigm of *pre-training+prompting*. With the help of class label information, we learn to generate class-specific and class-agnostic prompt graphs for fine-grained ID pattern mining. We also design effective losses to train the prompt generators and prevent trivial solutions. Extensive experiments conducted on ten benchmark datasets against very recent baselines demonstrate the effectiveness of DGP. For future work, a possible direction is to design more diverse prompt generators that can be adapted to different types of graphs.

ACKNOWLEDGMENT

The authors would like to thank the editor and anonymous reviewers for their valuable comments and suggestions, which helped to improve the quality and clarity of this paper.

REFERENCES

- [1] D. Dai and L. Van Gool, "Dark model adaptation: Semantic image segmentation from daytime to nighttime," in *Proc. Int. Conf. Intell. Transp. Syst.*, 2018, pp. 3819–3824.
- [2] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, "Concrete problems in ai safety," 2016, *arXiv:1606.06565*.
- [3] S. Liang, Y. Li, and R. Srikant, "Enhancing the reliability of out-of-distribution image detection in neural networks," in *Proc. Int. Conf. Learn. Representations*, 2018.
- [4] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," in *Proc. Int. Conf. Learn. Representations*, 2017.
- [5] Q. Wang et al., "Watermarking for out-of-distribution detection," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 15545–15557.
- [6] W. Zhou, F. Liu, and M. Chen, "Contrastive out-of-distribution detection for pretrained transformers," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2021, pp. 1100–1111.
- [7] Z. Shen et al., "Towards out-of-distribution generalization: A survey," 2021, *arXiv:2108.13624*.
- [8] Y. Liu, K. Ding, H. Liu, and S. Pan, "GOOD-D: On unsupervised graph out-of-distribution detection," in *Proc. ACM Int. Conf. Web Search Data Mining*, 2023, pp. 339–347.
- [9] Z. Li, Q. Wu, F. Nie, and J. Yan, "GraphDE: A generative framework for debiased learning and out-of-distribution detection on graphs," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 30277–30290.
- [10] Y. Guo, C. Yang, Y. Chen, J. Liu, C. Shi, and J. Du, "A data-centric framework to endow graph neural networks with out-of-distribution detection ability," in *Proc. 29th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2023, pp. 638–648.
- [11] Y. Hou et al., "Structural entropy guided unsupervised graph out-of-distribution detection," in *Proc. AAAI Conf. Artif. Intell.*, 2025, vol. 39, no. 16, pp. 17258–17266.
- [12] H. Junwei, Q. Xu, Y. Jiang, Z. Wang, Y. Sun, and Q. Huang, "HGOE: Hybrid external and internal graph outlier exposure for graph out-of-distribution detection," in *Proc. 32nd ACM Int. Conf. Multimedia*, 2024, pp. 1544–1553.
- [13] L. Wang et al., "GOODAT: Towards test-time graph out-of-distribution detection," in *Proc. AAAI Conf. Artif. Intell.*, 2024, vol. 38, no. 14, pp. 15537–15545.
- [14] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, and Y. Shen, "Graph contrastive learning with augmentations," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 5812–5823.
- [15] J. Xia, L. Wu, J. Chen, B. Hu, and S. Z. Li, "SimGRACE: A simple framework for graph contrastive learning without data augmentation," in *Proc. Int. Conf. World Wide Web*, 2022, pp. 1070–1079.

- [16] F. Xia et al., “Graph learning: A survey,” *IEEE Trans. Artif. Intell.*, vol. 2, no. 2, pp. 109–127, Apr. 2021.
- [17] Z. Zhang, P. Cui, and W. Zhu, “Deep learning on graphs: A survey,” *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 1, pp. 249–270, Jan. 2022.
- [18] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, and S. Y. Philip, “A comprehensive survey on graph neural networks,” *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 1, pp. 4–24, Jan. 2021.
- [19] H. Cai, V. W. Zheng, and K.C.-C. Chang, “A comprehensive survey of graph embedding: Problems, techniques, and applications,” *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 9, pp. 1616–1637, Sep. 2018.
- [20] P. Cui, X. Wang, J. Pei, and W. Zhu, “A survey on network embedding,” *IEEE Trans. Knowl. Data Eng.*, vol. 31, no. 5, pp. 833–852, May 2019.
- [21] D. Zhang, J. Yin, X. Zhu, and C. Zhang, “Network representation learning: A survey,” *IEEE Trans. Big Data*, vol. 6, no. 1, pp. 3–28, Mar. 2020.
- [22] J. Bruna, W. Zaremba, A. Szlam, and Y. LeCun, “Spectral networks and locally connected networks on graphs,” in *Proc. Int. Conf. Learn. Representations*, 2014.
- [23] M. Defferrard, X. Bresson, and P. Vandergheynst, “Convolutional neural networks on graphs with fast localized spectral filtering,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, vol. 29, pp. 3844–3852.
- [24] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 11313–11320.
- [25] W. Hamilton, Z. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, vol. 30, pp. 1025–1035.
- [26] P. Velickovic et al., “Graph attention networks,” in *Proc. Int. Conf. Learn. Representations*, 2018.
- [27] J. Gasteiger, A. Bojchevski, and S. Günnemann, “Predict then propagate: Graph neural networks meet personalized pagerank,” in *Proc. Int. Conf. Learn. Representations*, 2019.
- [28] X. Liu et al., “Self-supervised learning: Generative or contrastive,” *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 1, pp. 857–876, Jan. 2023.
- [29] Y. Liu et al., “Graph self-supervised learning: A survey,” *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 6, pp. 5879–5900, Jun. 2023.
- [30] L. Wu, H. Lin, C. Tan, Z. Gao, and S. Z. Li, “Self-supervised learning on graphs: Contrastive, generative, or predictive,” *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 4, pp. 4216–4235, Apr. 2023.
- [31] P. Veličković, W. Fedus, W. L. Hamilton, P. Liò, Y. Bengio, and R. D. Hjelm, “Deep graph infomax,” in *Proc. Int. Conf. Learn. Representations*, 2019.
- [32] Y. Zhu, Y. Xu, F. Yu, Q. Liu, S. Wu, and L. Wang, “Deep graph contrastive representation learning,” in *Proc. Int. Conf. Mach. Learn.*, 2020.
- [33] Y. Lu, X. Jiang, Y. Fang, and C. Shi, “Learning to pre-train graph neural networks,” in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, no. 5, pp. 4276–4284.
- [34] J. Qiu et al., “GCC: Graph contrastive coding for graph neural network pre-training,” in *Proc. 29th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2020, pp. 1150–1160.
- [35] W. Hu et al., “Strategies for pre-training graph neural networks,” in *Proc. Int. Conf. Learn. Representations*, 2020.
- [36] Z. Hu, Y. Dong, K. Wang, K.-W. Chang, and Y. Sun, “GPT-GNN: Generative pre-training of graph neural networks,” in *Proc. 29th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2020, pp. 1857–1867.
- [37] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, “Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing,” *ACM Comput. Surv.*, vol. 55, no. 9, pp. 1–35, 2023.
- [38] T. Brown et al., “Language models are few-shot learners,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 1877–1901.
- [39] X. L. Li and P. Liang, “Prefix-tuning: Optimizing continuous prompts for generation,” in *Proc. Assoc. Comput. Linguistics*, 2021, vol. 1, pp. 4582–4597.
- [40] B. Lester, R. Al-Rfou, and N. Constant, “The power of scale for parameter-efficient prompt tuning,” in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2021, pp. 3045–3059.
- [41] X. Liu et al., “P-Tuning: Prompt tuning can be comparable to fine-tuning across scales and tasks,” in *Proc. Assoc. Comput. Linguistics*, 2022, vol. 2, pp. 61–68.
- [42] X. Sun, H. Cheng, J. Li, B. Liu, and J. Guan, “All in one: Multi-task prompting for graph neural networks,” in *Proc. 29th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2023, pp. 2120–2131.
- [43] Z. Liu, X. Yu, Y. Fang, and X. Zhang, “GraphPrompt: Unifying pre-training and downstream tasks for graph neural networks,” in *Proc. Int. Conf. World Wide Web*, 2023, pp. 417–428.
- [44] H. Liu et al., “One for all: Towards training one graph model for all classification tasks,” in *Proc. Int. Conf. Learn. Representations*, 2024.
- [45] M. Sun, K. Zhou, X. He, Y. Wang, and X. Wang, “GPPT: Graph pre-training and prompt tuning to generalize graph neural networks,” in *Proc. 29th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 1717–1727.
- [46] Y. Zhu, J. Guo, and S. Tang, “SGL-PT: A strong graph learner with graph prompt tuning,” 2023, *arXiv:2302.12449*.
- [47] M. Stadler, B. Charpentier, S. Geisler, D. Zügner, and S. Günnemann, “Graph posterior network: Bayesian predictive uncertainty for node classification,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 18033–18048.
- [48] X. Zhao, F. Chen, S. Hu, and J.-H. Cho, “Uncertainty aware semi-supervised learning on graph data,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 12827–12836.
- [49] Q. Wu, Y. Chen, C. Yang, and J. Yan, “Energy-based out-of-distribution detection for graph neural networks,” in *Proc. Int. Conf. Learn. Representations*, 2023.
- [50] K. Ding, J. Li, N. Agarwal, and H. Liu, “Inductive anomaly detection on attributed networks,” in *Proc. Int. Joint Conf. Artif. Intell.*, 2021, pp. 1288–1294.
- [51] J. Jiang et al., “Anomaly detection with graph convolutional networks for insider threat and fraud detection,” in *Proc. IEEE Mil. Commun. Conf.*, 2019, pp. 109–114.
- [52] X. Wang, B. Jin, Y. Du, P. Cui, Y. Tan, and Y. Yang, “One-class graph neural networks for anomaly detection in attributed networks,” *Neural Comput. Appl.*, vol. 33, no. 18, pp. 12073–12085, 2021.
- [53] T. Zhao, C. Deng, K. Yu, T. Jiang, D. Wang, and M. Jiang, “Error-bounded graph anomaly loss for GNNs,” in *Proc. 29th ACM Int. Conf. Inf. Knowl. Manage.*, pp. 1873–1882, 2020.
- [54] L. Zhao and L. Akoglu, “On using classification datasets to evaluate graph outlier detection: Peculiar observations and new insights,” *Big Data*, vol. 11, no. 3, pp. 151–180, 2023.
- [55] R. Ma, G. Pang, L. Chen, and A. Van Den Hengel, “Deep graph-level anomaly detection by glocal knowledge distillation,” in *Proc. ACM Int. Conf. Web Search Data Mining*, 2022, pp. 704–714.
- [56] J. Sun et al., “Gradient-based novelty detection boosted by self-supervised binary classification,” in *Proc. AAAI Conf. Artif. Intell.*, 2022, vol. 36, no. 8, pp. 8370–8377.
- [57] J. Li, P. Chen, Z. He, S. Yu, S. Liu, and J. Jia, “Rethinking out-of-distribution (OOD) detection: Masked image modeling is all you need,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 11578–11589.
- [58] Y. Chen et al., “Learning causally invariant representations for out-of-distribution generalization on graphs,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 22131–22148.
- [59] Y. Sui, X. Wang, J. Wu, M. Lin, X. He, and T.-S. Chua, “Causal attention for interpretable and generalizable graph classification,” in *Proc. 29th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 1696–1705.
- [60] H. Li, Z. Zhang, X. Wang, and W. Zhu, “Learning invariant graph representations for out-of-distribution generalization,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, vol. 35, pp. 11828–11841.
- [61] T. Wu, H. Ren, P. Li, and J. Leskovec, “Graph information bottleneck,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 20437–20448.
- [62] C. Morris, N. M. Kriege, F. Bause, K. Kersting, P. Mutzel, and M. Neumann, “TUDataset: A collection of benchmark datasets for learning with graphs,” in *Proc. Int. Conf. Mach. Learn.*, 2020.
- [63] W. Hu et al., “Open graph benchmark: Datasets for machine learning on graphs,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, vol. 33, pp. 22118–22133.
- [64] X. Jiang et al., “Negative label guided OOD detection with pretrained vision-language models,” in *Proc. Int. Conf. Learn. Representations*, 2024.
- [65] Y. Zhang and L. Zhang, “AdaNeg: Adaptive negative proxy guided OOD detection with vision-language models,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, vol. 37, pp. 38744–38768.
- [66] F. Zeng, Z. Cheng, F. Zhu, H. Wei, and X.-Y. Zhang, “Local-prompt: Extensible local prompts for few-shot out-of-distribution detection,” in *Proc. Int. Conf. Learn. Representations*, 2025.
- [67] Y. Wu, R. Yu, X. Cheng, Z. He, and X. Huang, “Pursuing feature separation based on neural collapse for out-of-distribution detection,” in *Proc. Int. Conf. Learn. Representations*, 2025.
- [68] W. Chen, R. A. Yeh, S. Mou, and Y. Gu, “Leveraging perturbation robustness to enhance out-of-distribution detection,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2025, pp. 4724–4733.
- [69] M. Chen, Z. Wei, Z. Huang, B. Ding, and Y. Li, “Simple and deep graph convolutional networks,” in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 1725–1735.

- [70] Y. You, T. Chen, Y. Shen, and Z. Wang, "Graph contrastive learning automated," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 12121–12132.
- [71] S. Suresh, P. Li, C. Hao, and J. Neville, "Adversarial graph augmentation to improve graph contrastive learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 15920–15933.
- [72] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?," in *Proc. Int. Conf. Learn. Representations*, 2019.
- [73] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Representations*, 2015.
- [74] E. Zisselman and A. Tamar, "Deep residual flow for out of distribution detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 13994–14003.
- [75] S. Vernekar, A. Gaurav, V. Abdelzad, T. Denouden, R. Salay, and K. Czarnecki, "Out-of-distribution detection in classifiers via generation," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019.
- [76] J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in *Proc. Int. Conf. Mach. Learn.*, 2006, pp. 233–240.



Cheng Yang received the BE degree and PhD degrees from Tsinghua University in 2014 and 2019, respectively. He is currently an associate professor with the Beijing University of Posts and Telecommunications. He has authored or coauthored more than 40 top-level papers in international journals and conferences including *IEEE Transactions on Knowledge and Data Engineering*, *ACM TOIS*, *Proc. 29th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, and *Proc. Assoc. Comput. Linguistics*. His research interests include data mining and natural language processing.



Yu Hao is currently a Student with the Beijing University of Posts and Telecommunications. Her research interests include graph neural networks, machine learning, and large language model.



Qi Zhang received the master's degree from the Beijing University of Posts and Telecommunications. She is currently a Network Optimization engineer with China Mobile Group Shaanxi Company, Ltd. Her research interests include LTE wireless network optimization, graph neural networks, and Big Data mining.



Chuan Shi (Senior Member, IEEE) received the BS degree from Jilin University in 2001, the MS degree from Wuhan University in 2004, and PhD degree from the ICT of Chinese Academic of Sciences in 2007. He is currently a professor with the Beijing University of Posts and Telecommunications. He has authored or coauthored 100 papers in refereed journals and conferences, such as *SIGProc.*, *29th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, *Proc. Int. Joint Conf. Artif. Intell.*, *Proc. 29th ACM Int. Conf. Inf. Knowl. Management*, *IEEE Transactions on Knowledge and Data Engineering*, *Proc. Int. Conf. World Wide WebJ*, and *ACM TIST*. His research interests include data mining, machine learning, and evolutionary computing.